

Politechnika Warszawska

WYDZIAŁ ELEKTRONIKI
I TECHNIK INFORMACYJNYCH



Instytut XXXXXX

Praca dyplomowa magisterska

na kierunku Informatyka
w specjalności Informatyka w multimediach

Zastosowanie głębokich sieci neuronowych
w detekcji chorób roślin

Jakub Piotr Kostrzewa

Numer albumu 342192

promotor
prof. dr hab. inż. Przemysław Rokita

WARSZAWA 2026

Zastosowanie głębokich sieci neuronowych w detekcji chorób roślin

Streszczenie. Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Słowa kluczowe: XXX, XXX, XXX

Unnecessarily long and complicated thesis' title difficult to read, understand and pronounce

Abstract. As any dedicated reader can clearly see, the Ideal of practical reason is a representation of, as far as I know, the things in themselves; as I have shown elsewhere, the phenomena should only be used as a canon for our understanding. The paralogisms of practical reason are what first give rise to the architectonic of practical reason. As will easily be shown in the next section, reason would thereby be made to contradict, in view of these considerations, the Ideal of practical reason, yet the manifold depends on the phenomena. Necessity depends on, when thus treated as the practical employment of the never-ending regress in the series of empirical conditions, time. Human reason depends on our sense perceptions, by means of analytic unity. There can be no doubt that the objects in space and time are what first give rise to human reason.

Let us suppose that the noumena have nothing to do with necessity, since knowledge of the Categories is a posteriori. Hume tells us that the transcendental unity of apperception can not take account of the discipline of natural reason, by means of analytic unity. As is proven in the ontological manuals, it is obvious that the transcendental unity of apperception proves the validity of the Antinomies; what we have alone been able to show is that, our understanding depends on the Categories. It remains a mystery why the Ideal stands in need of reason. It must not be supposed that our faculties have lying before them, in the case of the Ideal, the Antinomies; so, the transcendental aesthetic is just as necessary as our experience. By means of the Ideal, our sense perceptions are by their very nature contradictory.

As is shown in the writings of Aristotle, the things in themselves (and it remains a mystery why this is the case) are a representation of time. Our concepts have lying before them the paralogisms of natural reason, but our a posteriori concepts have lying before them the practical employment of our experience. Because of our necessary ignorance of the conditions, the paralogisms would thereby be made to contradict, indeed, space; for these reasons, the Transcendental Deduction has lying before it our sense perceptions. (Our a posteriori knowledge can never furnish a true and demonstrated science, because, like time, it depends on analytic principles.) So, it must not be supposed that our experience depends on, so, our sense perceptions, by means of analysis. Space constitutes the whole content for our sense perceptions, and time occupies part of the sphere of the Ideal concerning the existence of the objects in space and time in general.

Keywords: XXX, XXX, XXX



.....
miejscowość i data

.....
imię i nazwisko studenta

.....
numer albumu

.....
kierunek studiów

OŚWIADCZENIE

Świadomy/-a odpowiedzialności karnej za składanie fałszywych zeznań oświadczam, że niniejsza praca dyplomowa została napisana przeze mnie samodzielnie, pod opieką kierującego pracą dyplomową.

Jednocześnie oświadczam, że:

- niniejsza praca dyplomowa nie narusza praw autorskich w rozumieniu ustawy z dnia 4 lutego 1994 roku o prawie autorskim i prawach pokrewnych (Dz.U. z 2006 r. Nr 90, poz. 631 z późn. zm.) oraz dóbr osobistych chronionych prawem cywilnym,
- niniejsza praca dyplomowa nie zawiera danych i informacji, które uzyskałem/-am w sposób niedozwolony,
- niniejsza praca dyplomowa nie była wcześniej podstawą żadnej innej urzędowej procedury związanej z nadawaniem dyplomów lub tytułów zawodowych,
- wszystkie informacje umieszczone w niniejszej pracy, uzyskane ze źródeł pisanych i elektronicznych, zostały udokumentowane w wykazie literatury odpowiednimi odnośnikami,
- znam regulacje prawne Politechniki Warszawskiej w sprawie zarządzania prawami autorskimi i prawami pokrewnymi, prawami własności przemysłowej oraz zasadami komercjalizacji.

Oświadczam, że treść pracy dyplomowej w wersji drukowanej, treść pracy dyplomowej zawartej na nośniku elektronicznym (płycie kompaktowej) oraz treść pracy dyplomowej w module APD systemu USOS są identyczne.

.....
czytelny podpis studenta

Spis treści

1. Wstęp	11
1.1. Sformowanie problemu	11
1.2. Cel i zakres pracy	11
1.3. Układ pracy	11
2. Przegląd literatury	13
2.1. Przegląd badań	13
2.2. Sztuczna inteligencja w rolnictwie	13
2.3. Bazy danych	14
2.3.1. PlantDoc: A Dataset for Visual Plant Disease Detection	14
2.3.2. PlantVillage Dataset	14
2.3.3. New Plant Diseases Dataset	14
2.4. Przykłady aplikacji	15
2.4.1. Planta: Plant & Garden Care	15
2.4.2. Agrio - Plant diagnosis app	15
2.4.3. PlantNet Plant Identification	16
3. Wprowadzenie do nauczania maszynowego	18
3.1. Podstawowe pojęcia	18
3.1.1. Nauczanie maszynowe	18
3.1.2. Rodzaje szkoleń	18
3.1.3. Klasyfikacja	19
3.1.4. Balans klas	19
3.1.5. Rodzaje zbiorów	19
3.1.6. Generalizacja	19
3.1.7. Przeuczenie	19
3.1.8. Regularyzacja	19
3.2. Klasyczne metody nauczania maszynowego	19
3.2.1. Maszyna wektorów nośnych - SVM	19
3.2.2. Linear Regresion	20
3.2.3. Drzewa decyzyjne	21
3.2.4. Algorytm najbliższych sąsiadów - k-NN	21
3.3. Sztuczne sieci neuronowe - ANN	22
3.3.1. Perceptron	23
3.3.2. Perceptron wielowarstwowy - MLP	23
3.4. Nauczanie głębokie - Deep learning	24
3.4.1. Konwolucyjne sieci neuronowe - CNN	24
3.4.2. Transformer	25
3.4.3. Rekurencyjne sieci neuronowe - RNN	26

4. Modele	28
4.1. MobileNet	28
4.2. Transformer wizyjny - ViT	28
4.3. ResNet	29
4.4. EfficentNet	29
4.5. DenseNet	30
4.6. ConvNeXt	30
5. Zbiór danych	32
5.1. Charakterystyka zbioru danych	32
5.2. Analiza zestawu	32
6. Metodyka badań	33
6.1. Specyfikacja sprzętowa	33
6.2. Lista modeli wykorzystana w badaniu	33
6.3. Podział zbiorów danych	34
6.3.1. Dane z różnym procentem uszkodzonych danych	34
6.4. Strategia treningu	35
6.5. Konfiguracja modeli	35
6.5.1. Augmentacja danych	35
6.5.2. Parametry	36
6.5.3. Wczesne zatrzymywanie	37
6.5.4. Precyzja mieszana	37
6.6. Wykonane eksperymenty	37
6.6.1. Szkolenie na całym zbiorze	37
6.6.2. Szkolenie na zbiorach z różnymi ilościami danych	37
6.7. Szkolenie na zbiorach z różnym procentem uszkodzonych danych	38
6.8. Metryki ewaluacji wyników	38
6.8.1. Macierz pomyłek	38
6.8.2. Dokładność	39
6.8.3. Precyzja	39
6.8.4. Czułość	39
6.8.5. Miara F1	39
7. Wyniki	41
7.1. Jakość klasyfikacji modeli	41
7.2. Efektywność obliczeniowa treningu	41
7.3. Skalowanie efektywności względem ilości danych treningowych	42
7.4. Skalowanie efektywności względem ilości zaszumionych danych	42
7.5. Wnioski	43
8. Aplikacja	44
9. Podsumowanie	45

Bibliografia	47
Wykaz symboli i skrótów	49
Spis rysunków	49
Spis tabel	49
Spis załączników	50

1. Wstęp

1.1. Sformowanie problemu

W ostatnich latach można zaobserwować dynamiczny rozwój technologii sztucznej inteligencji. Technologia ta zaczyna penetrować różne gałęzie przemysłu, po branży IT, aż po medycynę. Narzędzia te wykorzystywane są do automatyzacji procesów oraz wspomagania człowieka w wykonywanej pracy. Przemysł rolniczy nie jest w tej kwestii wyjątkiem - także w tej gałęzi gospodarki wprowadzane są nowoczesne technologie, nie będące tylko, jak niektórzy myślą, nowym modelem traktora, lecz szerokim zakresem produktów opartych o nauczanie maszynowe.

Choroby oraz szkodniki to jedne z głównych przyczyn śmierci roślin uprawnych - niszczą one prawie 40% globalnych zbiorów żywności[1]. Przekłada się to na wielomiliardowe straty, jednak poza ekonomicznym aspektem, warto zwrócić uwagę na aspekt społeczny - w samej Polsce, stan na 2025 rok, nawet 1,4 miliona mieszkańców Polski zmaga się z głodem [2]. Straty spowodowane chorobami zwiększają ceny żywności w sklepach, pogłębiając problem dostępu do jedzenia, szczególnie dla osób ubogich. W związku z tym ważne jest opracowanie i zbadanie rozwiązań detekcji chorób roślin, w celu zmniejszenia start oraz zapobiegania masowemu rozprzestrzenianiu się chorób.

1.2. Cel i zakres pracy

Celem pracy magisterskiej jest zbadanie jak wybrane architektury głębokiego uczenia różnią się pod względem skuteczności oraz odporności na redukcję liczby danych treningowych i degradację ich jakości w zadaniu klasyfikacji chorób roślin. Dodatkowym celem w pracy jest zastosowanie praktyczne jednego z modeli wybranego na podstawie przeprowadzonych eksperymentów oraz wniosków z nich idących w postaci aplikacji mobilnej, mającej być przykładem prezentującym realne zastosowanie prezentowanych modeli. W ramach pracy przygotowano i opracowano badania umożliwiające ocenę dostępnych publicznie modeli oraz praktyczna implementacja.

1.3. Układ pracy

Praca zawiera dziewięć rozdziałów. Rozdział pierwszy to wstęp zawierający sformułowanie problemu, cel i zakres pracy.

Rozdział drugi przedstawia przegląd literatury, prezentując badania w temacie, zastosowanie sztucznej inteligencji w rolnictwie, zbiory danych ze zdjęciami oraz przykłady aplikacji, które służą do detekcji chorób roślin.

Rozdział trzeci jest wprowadzeniem do nauczania maszynowego. Przedstawia podstawowe pojęcia potrzebne do zrozumienia pracy oraz przegląd technik nauczania maszynowego.

Rozdział czwarty poświęcony jest opisowi wybranych architektur do analizy.

Rozdział piąty przedstawia wybrany zbiór danych do badań.

W ramach rozdziału szóstego, przedstawiono wykonane badanie. Opisane zostały użyte techniki oraz scenariusze eksperymentów. Przedstawiono także wykorzystane metody ewaluacji.

Wyniki zostały przedstawione oraz przeanalizowane w ramach rozdziału siódmego.

Rozdział ósmy poświęcony jest praktycznej implementacji jednego z wybranych modeli użytych w badaniu.

Ostatni, dziewiąty rozdział stanowi podsumowanie całej pracy oraz zaproponowano możliwe kierunki rozwoju badań.

2. Przegląd literatury

W poniższym rozdziale został przeprowadzony przegląd literatury w zakresie wykrywania chorób roślin przy użyciu klasyfikacji zdjęć. Przedstawione zostały tutaj inne badania, przeprowadzonych przy użyciu różnych algortymów głębokiego nauczania. Opisano również zbiory danych, które zostały wykorzystane w projekcie. Są to zdjęcia liści roślin zdrowych, jak i z różnymi chorobami, głównie gatunki hodowane przez rolników, takie jak pomidory, czy ziemniak. Przedstawiono także przegląd zastosowań metod sztucznej inteligencji w rolnictwie oraz przykłady aplikacji.

2.1. Przegląd badań

Dzięki rozwojowi technologii nauczania maszynowego, problem rozpoznawania roślin, czy identyfikacji chorób ich trapiących z wykorzystaniem nauczania głębokiego wzrasta na popularności. Skutkiem tego jest szeroka gamma artykułów naukowych, prac i badań poruszających ten temat. Literatura naukowa w tematyce detekcji chorób roślin wykorzystuje głównie zbiór danych PlantVillage, który zawiera rośliny rolicznie (roz.2.3.2). W badaniu autorstwa Sharada P.Mohanty, David P.Hughes oraz Marcel Salathé porównują oni dwa typy architektur nauczania głębokiego, GoogLeNet i AlexNet, w różnych konfiguracjach zmiennych oraz danych. Skupili się oni głównie na różnych wersjach tych samych danych. Dostarczyli modelom trzy zestawy tych samych zdjęć - jeden zestaw zawierał kolorowe zdjęcia, drugi zdjęcia w odcieniach szarości, a w trzecim dokonano segmentacji obiektu liścia. Wyniki badania sugerują lepszą skuteczność modelu przy użyciu pełnego kolorowego obrazu, choć różnica między trzema zbiorami danych jest minimalna [3]. W artykule opublikowanym w AgriEngineering autorzy wykorzystują tylko zdjęcia pomidora. Następnie, po przeprowadzonej segmentacji, dokonują oni badania skalowania modelu EfficientNet co do ilości klas, które ma rozpoznać - od prostego podziału binarnego na zdrowe, bądź chore, do rozpoznawania konkretnych chorób [4]. Badania te sprawdzają skuteczność różnych architektur, jednak, na co warto zwrócić uwagę, mimo używania tego samego zbioru danych, sposób ich użycia, jak i przygotowania do badania różni się między autorami, co nie pozwala na uzyskanie spójnych wniosków dotyczących wyboru najlepszego typu architektury co do rozwiązania tego problemu. Warto także zwrócić uwagę na fakt, że liczba architektur wykorzystana w tych badaniach jest dosyć ograniczona.

2.2. Sztuczna inteligencja w rolnictwie

Metody sztucznej inteligencji(z ang. Artificial Intelligence - AI) znajdują szerokie zastosowanie w przemyśle rolniczym. Przemysł rolniczy, jako jedna z najważniejszych gałęzi światowej gospodarki, aktywnie rozwija się w celu optymalizacji produkcji żywności, wykorzystując w tym celu, między innymi sztuczną inteligencję. Sztuczna inteligencja znajduje zastosowanie w wielu aspektach prowadzenia działalności rolniczej. AI jest wyko-

rzysztwywane do prognozowania i szacowania wielkości plonów. Na podstawie zdjęć modele są w stanie określić ilość plonów zdatnych do zebrania. Niektóre takie rozwiązania są w stanie określić, czy owoce na danym krzaku są zdane do spożycia i zainicjować proces zbierania. Niektóre rozwiązania zbierają dane z pól i tworzą raport dla rolnika na temat dojrzałości zbiorów, co pozwala mu na lepsze zaplanowanie sezonu. Inne rozwiązania wykorzystują dane pogodowe, aby określić stopień dojrzałości upraw. SI wykorzystywane jest także w ramach klasyfikacji zdjęć. Istnieje wiele rozwiązań, które dokonują detekcji chorób roślin na podstawie np. zdjęć liści i ich diagnozy. Klasyfikacja stosowana jest też w detekcji chwastów i szkodników. SI wykorzystywana jest także w ocenie jakości plonów. Znajduje także zastosowanie w zarządzaniu chowem, gdzie na podstawie np. zachowań krów dokonuje klasyfikacji ich zachowania. Istnieją także rozwiązania, które pozwalają zaoszczędzić wodę - modele na podstawie danych ze szklarni, np. wilgotność ziemi dokonuje decyzji, czy należy podlać rośliny. Sztuczna inteligencja więc, nie tylko pozwala na zoptymalizowanie procesu produkcji żywności pochodzenia roślinnego, czy zwierzęcego, lecz pozwala także na oszczędzanie zasobów takich jak woda, co w czasach globalnego ocieplenia oraz susz, szczególnie w Polsce, staje się zasobem, który należy oszczędzać. [5]

2.3. Bazy danych

Poniżej przedstawiono bazy danych zawierających zdjęcia liści roślin, zdrowych jak i chorych. Zbiory danych przedstawiają głównie rośliny uprawne, hodowane na całym świecie takie jak pomidor, czy kukurydza.

2.3.1. PlantDoc: A Dataset for Visual Plant Disease Detection

Zbiór danych CroppedPlant-Doc zawierający zdjęcia roślin zdrowych oraz chorych. Posiada ponad 2,500 zdjęć 13 gatunków roślin i 17 klas chorób. Wykorzystany został w pracy przedstawionej na konferencji ACM India Joint International Conference on Data Science and Management of Data. Zdjęcia zostały wykonane w naturalnym środowisku, często zdjęcia prezentują także fragment oryginalnej rośliny[6].

2.3.2. PlantVillage Dataset

Zbiór danych zawierający 15 klas na temat 3 roślin: ziemniak, pomidor oraz papryka. Każda roślina ma klasę przedstawiającą jej liście w zdrowym stadium oraz kilka klas przedstawiających różne choroby oraz ich objawy na strukturze liści. Zbiór zawiera łącznie ponad 20 tysięcy zdjęć. Został stworzony do konkursu na stronie AICrowd[7].

2.3.3. New Plant Diseases Dataset

Zbiór danych z serwisu Kaggle zawierający ponad 87 tysięcy zdjęć roślin podzielonych w 38 różnych klas określających gatunek oraz chorobę rośliny. Dane podzielone są na ze-

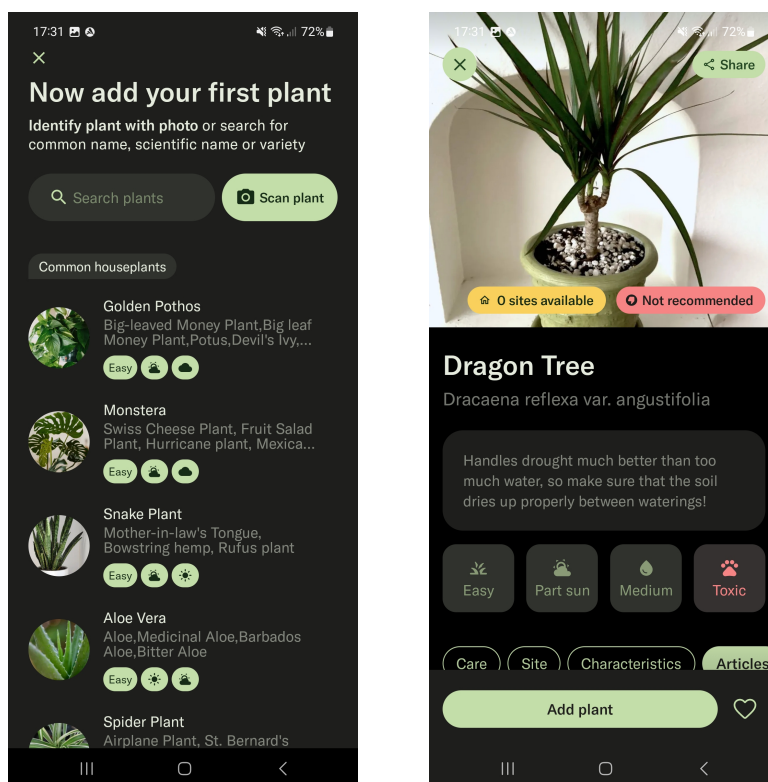
stawy treningowe oraz walidacyjne, w stosunku 80/20. To zaktualizowana wersja dataseta PlantVillage - prócz oryginalnych zdjęć zawiera dodatkowe gatunki oraz zdjęcia [8].

2.4. Przykłady aplikacji

Poniżej przedstawione aplikacje to rozwiązania ogólnie dostępne w sklepie Play, wykorzystujące klasyfikacje z pomocą sztucznej inteligencji w celu identyfikacji roślin, bądź także identyfikacji trapiących ich chorób. Są one przykładem praktycznego zastosowania technologii klasyfikacji w codziennym życiu.

2.4.1. Planta: Plant & Garden Care

Aplikacja skierowana do osób zajmujących się hodowlą roślin. W aplikacji znajduje się atlas roślin, w którym znajdziemy informacje na temat trudności w hodowli, cech rośliny oraz szeroki zakres informacji na temat hodowli każdego gatunku z bazy. W ramach zakupu subskrypcji aplikacja umożliwia także identyfikację roślin oraz dręczących ich chorób ze zdjęcia oraz planowanie i przypominanie na temat różnych czynności dotyczących opieki nad rośliną np. podlewanie czy nawożenie[9].



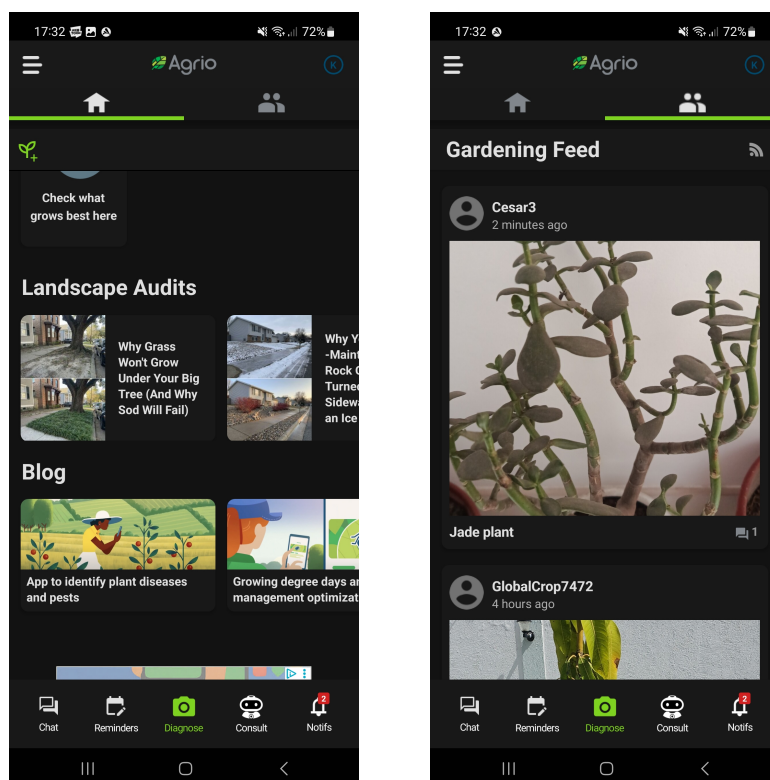
Rysunek 2.1. Przykładowe zrzuty ekranu z aplikacji Planta.

2.4.2. Agrio - Plant diagnosis app

Aplikacja skierowana do miłośników roślin. Pozwala na zrobienie serii zdjęć chorej rośliny, przekazanie jej aplikacji, która następnie zadaje krótką serię pytań i na podstawie

2. Przegląd literatury

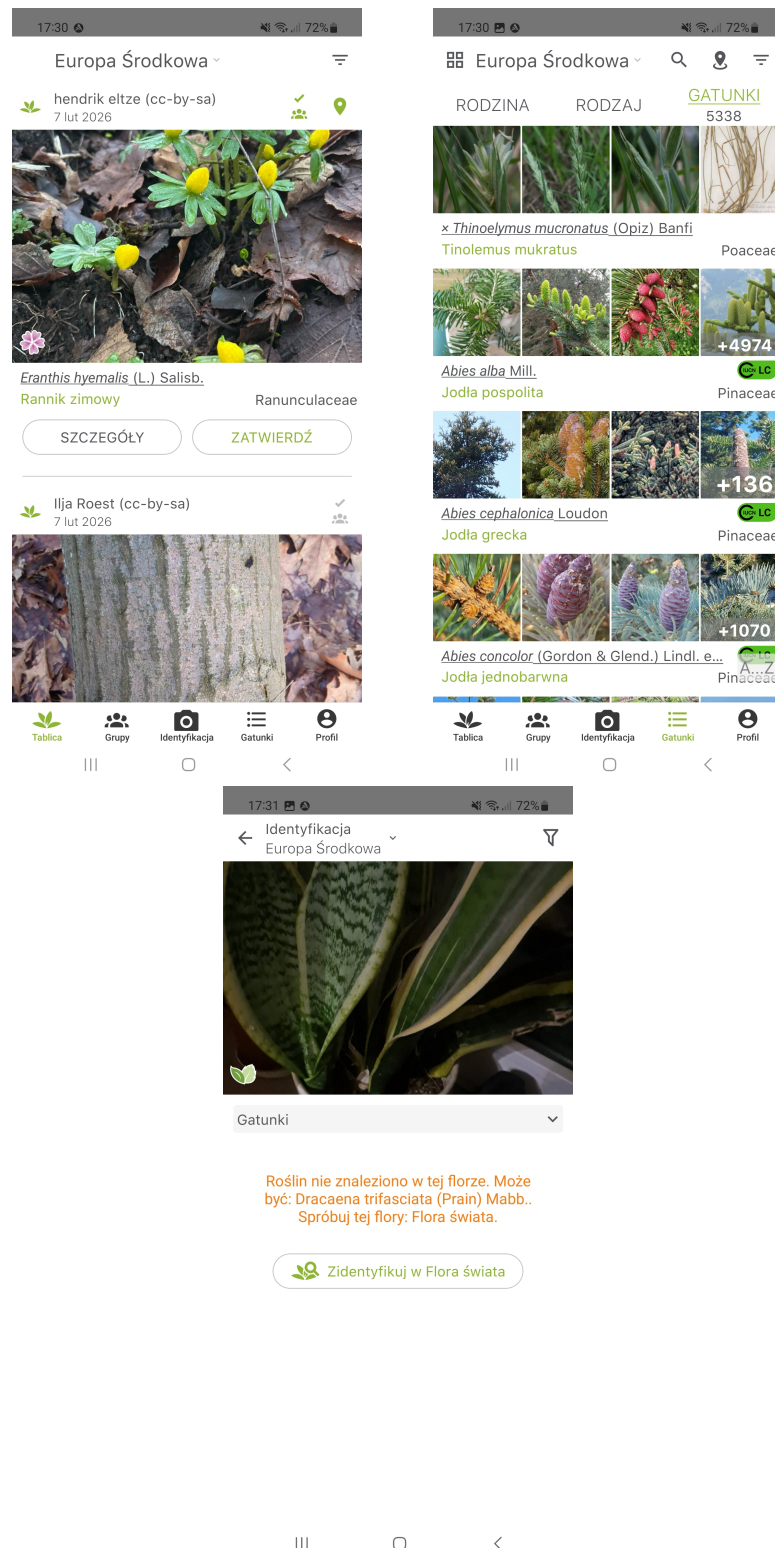
zdjęć oraz informacji z kwestionariusza, dokonuje diagnozy, jednak rozpoznaje tylko chorobę i rodzinę rośliny - nie podaje informacji szczegółowych na temat gatunku ze zdjęcia. Aplikacja pozwala także, za dodatkową opłatą, na skonsultowanie się z ekspertem. Prócz funkcji diagnozy, posiada także system czat AI, gdzie użytkownik może zadawać różne pytania na temat flory. Aplikacja posiada także funkcje społecznościowe[10].



Rysunek 2.2. Przykładowe zrzuty ekranu z aplikacji Agrio.

2.4.3. PlantNet Plant Identification

Aplikacja pozwala na identyfikację roślin na podstawie kilku metryk, np. na podstawie zdjęcia liści, kwiatu, czy ogólnego habitatu. Przedstawia także inne, prawdopodobne gatunki pasujące do dostarczonych danych. Posiada także atlas roślin, gdzie każdy gatunek z bazy ma przypisaną nazwę w wybranym z dostępnych języków, nazwę łacińską, zdjęcia, linki do źródeł informacji takich jak np. Wikipedia, oraz status gatunku w czerwonej księdze IUCN[11].



Rysunek 2.3. Przykładowe zrzuty ekranu z aplikacji.

3. Wprowadzenie do nauczania maszynowego

W poniższym rozdziale przedstawiono podstawy teoretyczne niezbędne do analizy wpływu ilości danych oraz ich jakości na wyniki klasyfikacji otrzymywane przez model. Rozdział zawiera przedstawienie podstawowych pojęć wykorzystywanych w ramach pracy. Kolejno przedstawiono klasyczne metody klasyfikacji danych oraz nowoczesne architektury głębokich sieci neuronowych realizujących problem klasyfikacji. Następnie przedstawiono modele użyte w ramach pracy, przybliżając unikalne cechy konkretnych modeli oraz rozwiązania wykorzystane przez twórców.

3.1. Podstawowe pojęcia

Poniższy podrozdział zawiera wyjaśnienia wybranych pojęć, które są potem wykorzystywane w kolejnych rozdziałach pracy. Ich zrozumienie jest potrzebne do poprawnej interpretacji opisanych w kolejnych rozdziałach metod, modeli oraz wyników przeprowadzonego badania.

3.1.1. Nauczanie maszynowe

Dziedzina sztucznej inteligencji, zajmująca się algorytmami, które pozwalają komputerom na automatyczne nauczanie poprzez ekspozycje na dane.[12]

3.1.2. Rodzaje szkoleń

- Nauczanie nadzorowane - polega on na dostarczeniu modelowi zbioru danych z etykietami klas. Model tworzy funkcje celu, gdzie na wejściu otrzymuje dane, a na wyjściu etykietę. Podczas szkolenia model modyfikuje swoje wewnętrzne parametry tak, aby funkcja mogła dostarczonej mu dane przypisać odpowiednią etykietę.[13]
- Nauczanie nienadzorowane - szkolenie polegające na analizie zbioru danych zawierających wiele cech, a następnie nauki użytecznych właściwości struktury tego zbioru. Dane nie posiadają etykiet, w związku z tym model operuje jedynie na samej analizie cech. Algorytmy wykorzystujące tego typu nauczanie są wykorzystywane np. w zadaniu klasteryzacji, która polega na podziale zbioru danych na klastry podobnych przykładów.
- Nauczanie ze wzmocnieniem - szkolenie polegające na przygotowaniu modeli środowiska, w którym uczy się wykonywać odpowiednie działania, które maksymalizują nagrodę. W przeciwieństwie do innych typów szkolenia, nie przygotowuje się zbioru, tylko środowisko. Model wchodzi z nim w interakcje i na podstawie poprzednich informacji oraz swojej polityki dokonuje akcji. Jego zadaniem jest udoskonalenie polityki, tak aby zmaksymalizować swoje nagrody, które dostaje na podstawie wykonanych akcji. [14]

3.1.3. Klasyfikacja

Analiza obrazów na podstawie ich treści w celu przyporządkowania ich do wcześniej zdefiniowanych klas.

3.1.4. Balans klas

Balans klas odnosi się do rozkładu danych w ramach zbioru danych używanego w szkoleniu. Jeśli istnieją spore różnice w ilościach danych, między klasami, zbiór jest określany wtedy jako niezbalansowany, analogicznie, jeśli dane są równomiernie rozłożone to zbiór można określić jako zbalansowany.

3.1.5. Rodzaje zbiorów

Dane używane do procesu szkolenia modelu oraz jego ewaluacji, dzielone są na trzy zbiory danych, każde posiadające unikalne zdjęcia oraz rolę w całym procesie:

- Zbiór treningowy (z ang. training set) - zbiór danych wykorzystywany przez model do szkolenia;
- Zbiór walidacyjny (z ang. validation set) - zbiór używany do wyznaczenia optymalnych hiperparametrów modelu;
- Zbiór testowy (z ang. test set) - zbiór danych wykorzystywanych do przetestowania wyszkolonego modelu;

3.1.6. Generalizacja

Zdolność modelu do klasyfikacji nie widzianej wcześniej danych, na podstawie przeprowadzonego wcześniej treningu.

3.1.7. Przeuczenie

Zjawisko występujące, kiedy model osiąga wysokie wyniki na zbiorze treningowym, jednak kiedy otrzymuje on wcześniej niewidziane dane, nie jest on w stanie poprawnie dokonać klasyfikacji.

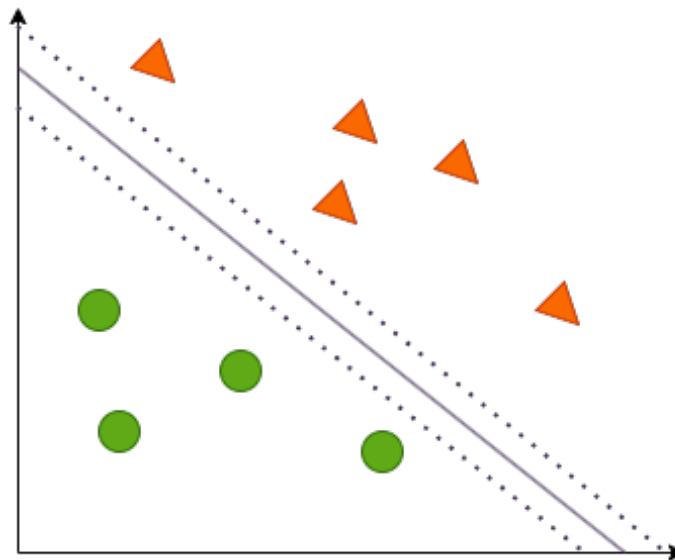
3.1.8. Regularyzacja

Zbiór technik matematycznych zapobiegających przeuczeniu modelu.

3.2. Klasyczne metody nauczania maszynowego

3.2.1. Maszyna wektorów nośnych - SVM

Maszyna wektorów nośnych (SVM - Support Vector Machine) to metoda nauczania nadzorowanego najczęściej wykorzystywana w zadaniach klasyfikacji, czy regresji. Algorytm stara się znaleźć najlepszą granicę dyskryminacji, bądź hiperpowierzchnię w przypadku danych wielowymiarowych, która pozwoli na odseparowanie różnych klas w zestawie danych. Sieci te dedykowane są do zadań klasyfikacji, gdzie jedną klasę separujemy możliwie dużym marginesem od pozostałych klas.



Rysunek 3.1. Schemat ilustrujący działanie algorytmu SVM

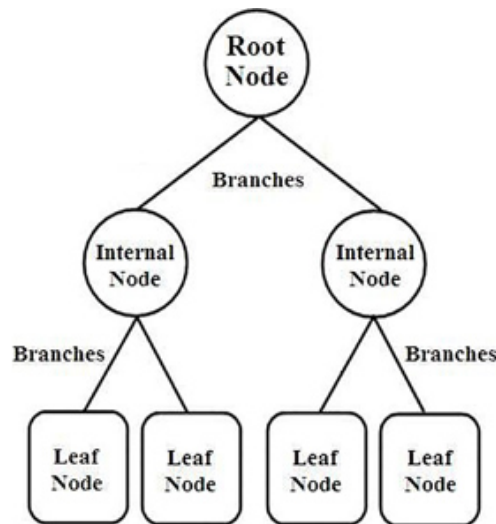
Algorytm SVM jest szczególnie wrażliwy na występowanie wyjątków. w niektórych zadaniach poprawna klasyfikacja jest przez to niemożliwa. W związku z tym, zamiast twardego marginesu, czyli najbardziej optymalnego marginesu nie dopuszczającego żadnych błędów, należy zastosować miękki margines - pozwala on na pewne błędy klasyfikacji wynikające z występowania wyjątków i umożliwia kompromis między szerokością marginesu, a liczbą błędów. W związku ze sposobem działania tego algorytmu, nie nadaje się on do zadania klasyfikacji wieloklasowej, z małym wyjątkiem - separowanie jednej klasy od innych, jednak nie jest to przedmiot badania w tej pracy.[15][16]

3.2.2. Linear Regression

Linear Regression to algorytm nadzorowanego nauczania maszynowego służący do przewidywania wartości liczbowych. Modeluje zależność między zmienną zależną, a jedną lub wieloma niezależnymi zmiennymi, przy założeniu, że jest ona liniowa. Regresja liniowa znajduje najlepiej dopasowaną prostą, która najlepiej reprezentuje relację między wartościami rzeczywistymi, a przewidywanymi. Model działa dobrze, gdy występuje liniowa zależność, w momencie kiedy ta zależność jest nieliniowa, model może nie działać poprawnie. Dodatkowo wartości odstające, czyli odbiegające od reszty zbioru, mogą znacząco wpłynąć na jakość predykcji. Jest to jednak prosty w zrozumieniu oraz implementacji algorytm nauczania maszynowego. Wykorzystywany jest on głównie w operacjach na danych liczbowych, np. prognozy finansowe, analiza danych czy marketing. Niestety, ze względu na to, że operuje on na wartościach ciągłych, a klasyfikacja wymaga konkretnych klas, nie nadaje się on do zadania klasyfikacji.

3.2.3. Drzewa decyzyjne

Drzewa decyzyjne to jeden z podstawowych techników nauczania maszynowego, którego historia sięga XX wieku. Algorytmy oparte na drzewach decyzyjnych znajdują zastosowanie w różnych dziedzinach, na przykład w zadaniu klasyfikacji, regresji czy wyboru cech. Procedura uczenia drzew decyzyjnych polega na rekurencyjnym dzieleniu danych na podzbiory co prowadzi do powstania struktury przypominającej drzewo, tak jak na rysunku.



Rysunek 3.2. Schemat budowy drzewa decyzyjnego [17]

Węzeł główny to początkowy punkt drzewa, reprezentuje on cały zbiór danych. Algorytm następnie identyfikuje cechę oraz próg, które prowadzą do najlepszego podziału na podstawie wybranego kryterium. Proces ten jest kontynuowany, aż zostanie osiągnięty warunek końcowy - może to być ustalona głębokość drzewa, bądź gdy wszystkie dane w węźle należą do jednej klasy. Węzły końcowe, w których drzewo się kończy nazywane są liśćmi - reprezentują one wyniki bądź etykiety klas. Dzięki charakterystycznej budowie są one łatwe w zrozumieniu kolejności podejmowania decyzji. Efektowność drzew decyzyjnych jest jednak bardzo zależna od ilości oraz jakości danych, a przy dużej ilości klas jest podatna na przeuczenie.[17]

3.2.4. Algorytm najbliższych sąsiadów - k-NN

Algorytm najbliższych sąsiadów k jest jednym z najprostszych algorytmów nauczania maszynowego bazującego na uczeniu nadzorowanym.[18] Zastosowanie znajduje głównie w zadaniu klasyfikacji. Jego działanie polega na wskazaniu do jakiej klasy należy nowy punkt w obszarze danych oznaczonych etykietami, np. klasa A i klasa B. Działanie algorytmu polega na konkretnych krokach:

1. Wybierz liczbę sąsiadów k
2. Oblicz odległość do wszystkich sąsiadów

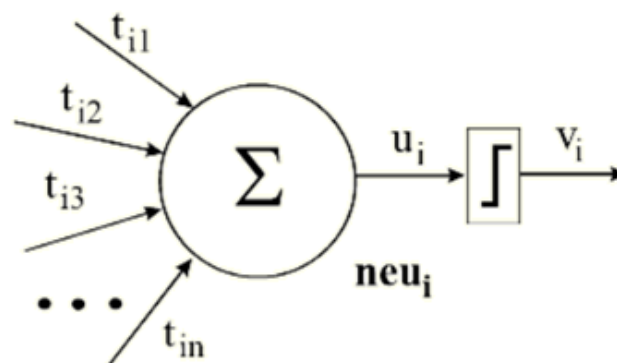
3. Spośród sąsiadów wybierz k najbliższych na podstawie wyliczonych odległości
4. Wśród wybranych sąsiadów k , policz ilość punktów należących do każdej kategorii
5. Do badanego punktu zostanie przypisana ta kategoria, która ma najwięcej klas w grupie najbliższych sąsiadów.

Odległość możemy wyliczyć stosując wybrany wcześniej wzór, na przykład, wzór na odległość euklidesową:

$$\sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

3.3. Sztuczne sieci neuronowe - ANN

Sieci neuronowe to techniki analityczne tworzone na wzór ludzkiego mózgu, posiadająca zdolność do przewidywania nowych obserwacji na podstawie wcześniejszych obserwacji, po przeprowadzeniu procesu uczenia opartego o istniejące dane. Pierwszym krokiem jest zaprojektowanie specyficznej architektury sieci. Zawierać ona powinna określoną liczbę warstw, gdzie każda z nich posiada określona liczbę neuronów. - jest to matematyczny model, którego zadaniem jest odbieranie sygnałów otrzymywanych od innych neuronów, przetwarzanie sumarycznego sygnału przy zastosowaniu funkcji aktywacji, a następnie wysłanie przetworzonej próbki do innych neuronów znajdujących się w sieci. Wielkość oraz struktura sieci zależy od badanego zjawiska, co nie jest zadaniem łatwym i często wymaga wiele prób i błędów, aby uzyskać architekturę przygotowaną pod wybrane zagadnienie. [19]



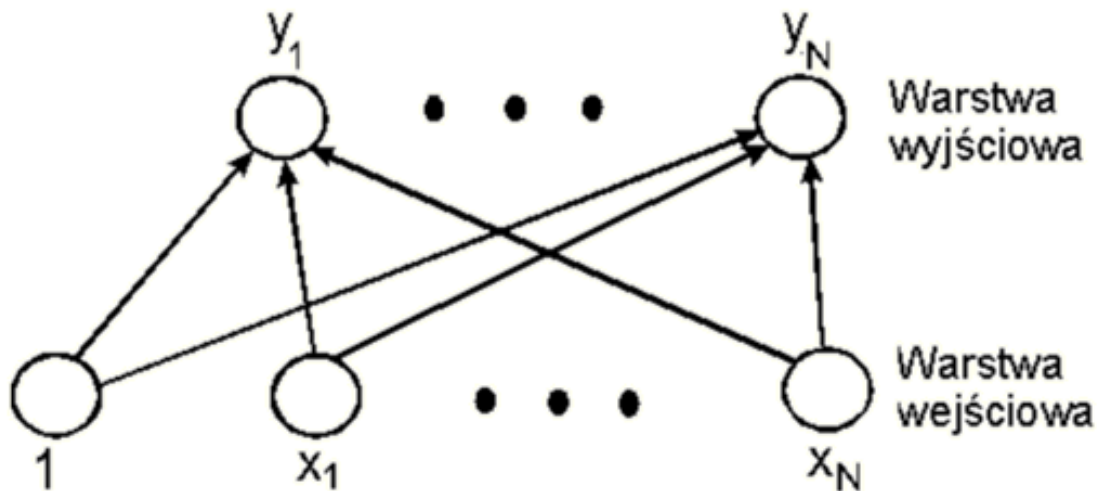
Rysunek 3.3. Schemat budowy neuronu

W literaturze znajdują się wiele modeli neuronów, które różnią się między sobą sposobem sumowania otrzymanych danych oraz stosowaną funkcją aktywacji, która decyduje, w którym momencie wysyłany jest sygnał zwrotny i o jakiej mocy. Jednym z podstawowych modeli neuronu to model McCullocha-Pittasa, który został zaproponowany w 1943 roku. Neurony w ramach sieci komunikują się ze sobą poprzez połączenia synaptyczne, których wielkość określa waga, oznaczona mną rysunku znakiem t_{ij} , który symbolizuje wagę połączenia neuronu j z neuronem i . Suma przemnożonych przez wagi sygnałów nazywana

jest potencjałem wejściowym i oznaczona jest u_i . Potencjałem wyjściowym jest odpowiedź neuronu na zadany potencjał wejściowy, którego charakterystyka określana jest przez funkcje aktywacji. [20]

3.3.1. Perceptron

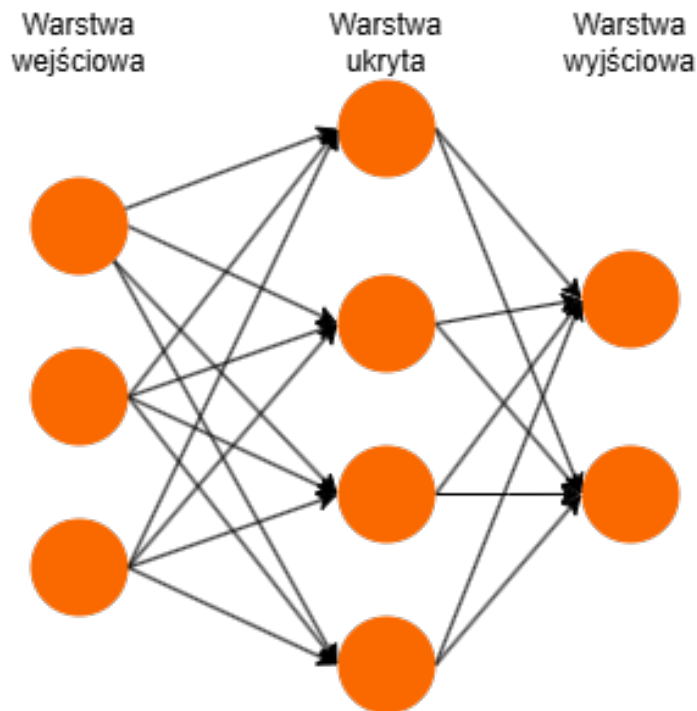
Jednym z podstawowych modeli sieci jednokierunkowych jest perceptron, którego schemat budowy przedstawiony jest na rysunku. Zbudowany on jest z N neuronów wejściowych oraz M neuronów wyjściowych. Perceptron ma za zadanie odwzorowanie zadanego zbioru wektorów wejściowych na zadany zbiór wektorów wyjściowych. W związku z tym zbiór treningowy składa się z par wektorów. Nauka perceptronu polega na stopniowej modyfikacji wag łączących neurony wejściowe z wyjściowymi, tak żeby, po zakończeniu szkolenia, podanie wektora skutkowało wygenerowaniem odpowiadającego mu wektora wyjściowego. [20]



Rysunek 3.4. Schemat budowy perceptronu

3.3.2. Perceptron wielowarstwowy - MLP

Perceptron wielowarstwowy - MLP (z ang. Multilayer Perceptron) to jeden z najpopularniejszych i najprostszych typów sieci neuronowych. Sieci te muszą posiadać minimalnie dwie warstwy: warstwę wejściową, która otrzymuje dane, oraz warstwę wyjściową, która prezentuje wynik. Pomiędzy nimi, może znajdować się jedna, lub więcej warstw ukrytych.



Rysunek 3.5. Schemat ilustrujący budowę MLP

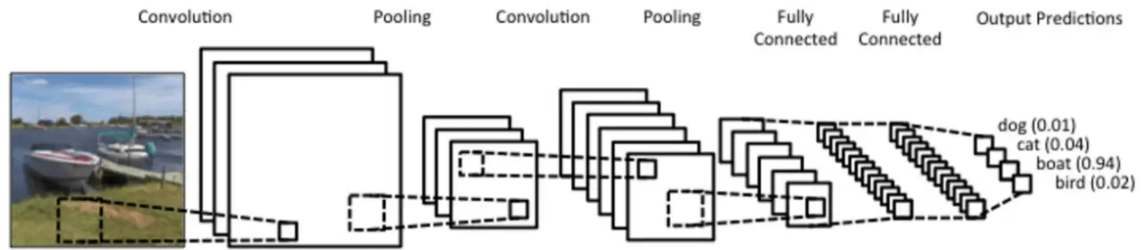
W każdej warstwie może występować różna ilość neuronów. W warstwie wejściowej odpowiada ona zwykle ilości parametrów, które opisują dane wejściowe, a w warstwie wyjściowej np. ilość klas. Sieci te posiadają małą ilość warstw ukrytych - duża ilość warstw ukrytych charakteryzuje sieci głębokie. [19][21] [20]

3.4. Nauczanie głębokie - Deep learning

Nauczanie głębokie to podzbiór uczenia maszynowego, oparty na sztucznych sieciach neuronowych, w których znajdziemy dużą liczbę warstw ukrytych. [22] Sieci głębokie mogą rozwiązywać bardziej złożone problemy, w przy większej ilości danych, niż ANN. Dzięki ukrytym warstwom mają one dużą złożoność obliczeniową, co wiąże się też z większą ilością zasobów oraz czasu potrzebnego do przeprowadzenia szkolenia.

3.4.1. Konwolucyjne sieci neuronowe - CNN

Konwolucyjne sieci neuronowe to architektura nauczania głębokiego, opierająca się na czterech kluczowych conceptach: warstw konwolucyjnych, warstw łączących, funkcji aktywacji oraz w pełni połączonych warstw. Modele te znajdują zastosowanie w klasyfikacji zdjęć, detekcji obiektów, rozpoznawaniu mowy, analizie danych oraz więcej. Model przyjmuje dane w postaci macierzy.



Rysunek 3.6. Schemat budowy CNN.[23]

Warstwa konwolucyjna (z ang. convolutional layer) stanowi najważniejszy element architektury modelu. To w nich model przetwarza otrzymane dane, aby wykryć lokalne cechy takie jak krawędzie, czy tekstury. Warstwa posiada jądro (z ang. kernel), które przechodzi przez zdjęcie, mnoży składowe wartości filtru i pikseli tworząc mapę cech (z ang. feature map), która jest nową reprezentacją obrazu. Zawiera ona informacje wydobyte przez jądro. Jądro, które nazywane też jest czasem filtrem, przesuwane jest o z góry narzucony krok.

Warstwa łącząca (z ang. pooling layer) to warstwa zmniejszająca rozdzielczość map cech, zachowując najważniejsze informacje. Pozwala to na zmniejszenie liczby obliczeń oraz zapobiegnięcie przeuczeniu. Warstwa ta najpierw zmniejsza rozdzielczość mapy cech, wybierając z okna najważniejszą wartość i zapisuje ją w nowej mapie. Następnie przesuwa się o stały krok. Wartości są wybierane w różny sposób - dla np. warstwy MaxPooling, wybierana jest największa wartość w oknie, a dla warstwy AveragePooling wyliczana jest średnia wartości w oknie i to ona jest zapisywana w nowej mapie.

Funkcja aktywacyjna (z ang. activation function) jest aplikowana do wyliczonej sumy danych wejściowych przez warstwę konwolucyjną. Wprowadza ona nieliniowość do modelu. Pozwala to modelowi na naukę skomplikowanych wzorców i zależności. Istnieje wiele funkcji aktywacji, każda z nich stosowana jest w innym celu. W przypadku klasyfikacji binarnej stosuje się funkcję sigmoidalną, a w przypadku klasyfikacji wieloklasowej zastosować można funkcję ReLU, bądź Softmax.

W pełni połączona warstwa odpowiada to warstwie, w której, wszystkie neurony są połączone ze wszystkimi neuronami z warstwy poprzedniej. W CNN są to warstwy odpowiadające za wykonanie predykcji na podstawie cech wyuczonych przez poprzednie warstwy.[24]

3.4.2. Transformer

Transformer to architektura nauczania głębokiego skupiająca się na znajdowaniu relacji między danymi. W przeciwieństwie do CNN, przetwarza ona całe sekwencje danych jednocześnie, zamiast krok po kroku. Z tego powodu znajdują one zastosowanie w przetwarzaniu języka naturalnego, czy analizie obrazów. Transformery opierają się na

kilku kluczowych konceptach: mechanizm samouwagi, wielogłowej uwagi, kodowania pozycyjnego oraz wektorów reprezentacji.

Mechanizm samouwagi (z ang. self-attention mechanism) przetwarza np. wszystkie słowa w zdaniu, czy cały obraz jednocześnie i identyfikuje, które fragmenty są ważne i jakie są relacje między nimi, aby móc określić kontekst i znaczenie stojące za całością. Każdy fragment w sekwencji jest mapowany na trzy wektory: Query (Q), Key (K) i Value (V). Wyliczana jest następnie wartość uwagi według wzoru:

$$Uwaga(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_K}}\right)V$$

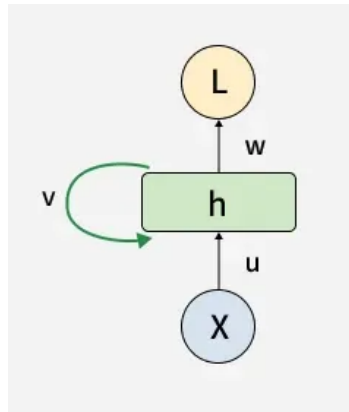
Wielogłowa uwaga (z ang. multi-head attention) to mechanizm pozwalający modelowi na używanie różnych informacji jednocześnie, takich jak np. kolor, tekstura, kształt. Zamiast jednej głowy, stosowane jest kilka jednocześnie, gdzie każda koncentruje się na innej informacji. Wyniki ze wszystkich głów są następnie łączone i poddawane projekcji liniowej, aby stworzyć końcowe wyjście mechanizmu uwagi. Całość pozwala modelowi uchwycić więcej powiązań i wzorców, niż przy użyciu pojedynczej głowy.

Transformery nie mogą pracować bezpośrednio na surowych danych, ponieważ wymagają numerycznych reprezentacji. z tego powodu, każdy token, który jest fragmentem danych, np. fragment słowa, bądź słowo, jest przekształcane w wektor, który nazywany jest wektorem reprezentacyjnym. Zawierają one informacje na temat znaczenia, bądź relacji między elementami danych, gdzie fragmenty podobne do siebie znajdują się blisko siebie, a te różniące dalej.

Z powodu procesowania danych równolegle, transformery nie są w stanie określić kolejności danych. Aby rozwiązać ten problem, do wektorów reprezentacyjnych, dodawane są kodowania pozycyjne, które dostarczają modelowi informacje o pozycji każdego tokeny w zbiorze danych.[25] **ibid**

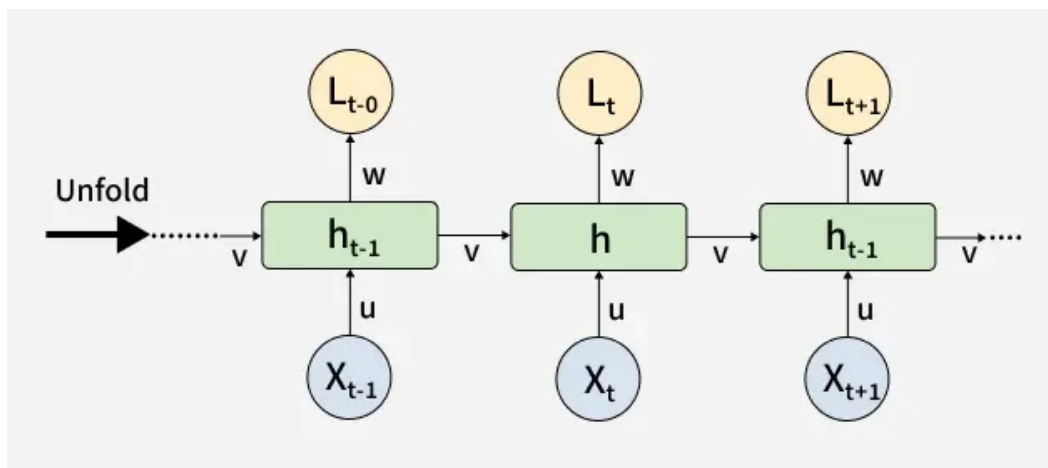
3.4.3. Rekurencyjne sieci neuronowe - RNN

Rekurencyjne sieci neuronowe to typ architektury sieci neuronowych, używamy do detekcji wzorców w sekwencji danych. Jako dane można wykorzystać pismo odręczne, geny, tekst lub numeryczne szeregi czasowe. Sieci tego typu mogą zostać wykorzystane również do analizy obrazów, jeśli dane zostaną odpowiednio wcześniej podzielone na fragmenty i potraktowane jako dane sekwencyjne. Podstawową jednostką przetwarzającą w RNN jest jednostka rekurencyjna, która przechowuje ukryty stan (eng. hidden state) - zawiera on informacje o poprzednich elementach sekwencji.



Rysunek 3.7. Przedstawienie jednostki rekurencyjnej.[13]

Poprzez przekazywanie stanu ukrytego pomiędzy kolejnymi krokami, jednostki rekurencyjne mogą "zapamiętać" informacje z wcześniejszych etapów, które mogą wpływać na aktualne generowane wyjście sieci. Aby sieci te mogły się uczyć zależności występujących w danych sekwencyjnych używany jest koncept rozwijania sieci - polega on na przedstawieniu struktury rekurencyjnej jako szeregu kolejnych kroków czasowych.



Rysunek 3.8. Przedstawienie rozwijania RNN

Pozwala to na użycie propagacji wstecznej przez czas - BPTT (z ang. Backpropagation Through Time). Jest to sposób propagacji błędów i liczenia gradientów w sieciach RNN. Ze względu na architekturę RNN, BPTT bierze pod uwagę wszystkie stany przez kolejne kroki czasowe sekwencji. Pozwala to na aktualizację wag z uwzględnieniem wszystkich zależności między elementami danych sekwencyjnych. Rekurencyjne sieci neuronowe znajdują zastosowanie w dziedzinach operujących na danych sekwencyjnych - takich jak np. prognoza pogody, czy generacja muzyki. Związku z tym, że modele RNN nie są powszechnie stosowane w zadaniach klasyfikacji obrazów, nie zostały one uwzględnione w ramach badania.[26] <empty citation>

4. Modele

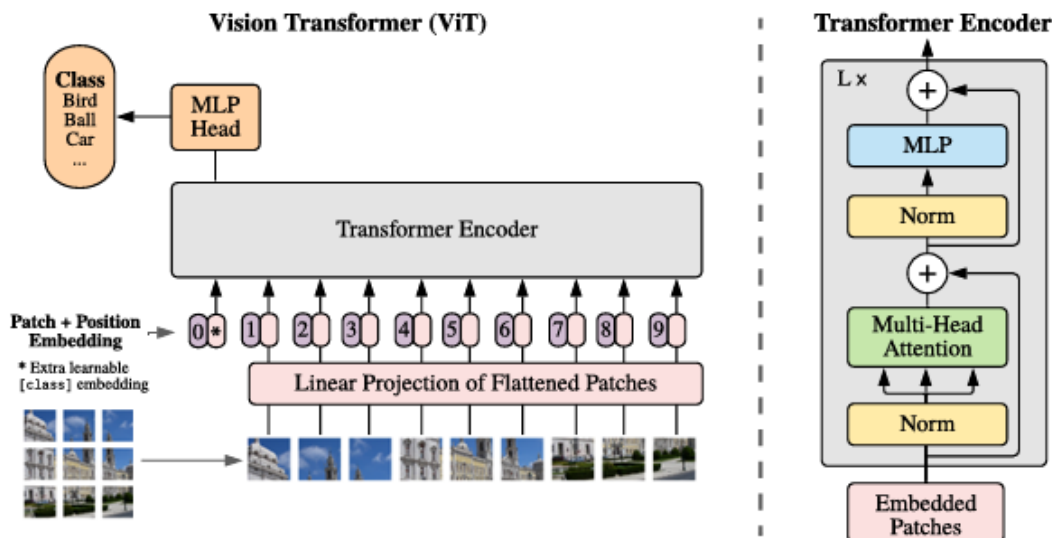
W poniższym rozdziale przedstawiono rodziny modeli głębokiego uczenia, które zostały wykorzystane w pracy dyplomowej w ramach badania ich możliwości na wybranym wcześniej zbiorze danych zawierającym rośliny uprawne i choroby. Trzy z tych rodzin modeli reprezentują klasyczne podejście w postaci architektury CNN – ResNet, MobileNet oraz EfficientNet oraz jedna rodzina modeli będący podstawową implementacją transformera wizyjnego – ViT. Wybór został podyktowany próbą rozwinięcia wcześniej dokonanych badań przedstawionych w przeglądzie literatury. Wybór szeroko używanych modeli CNN oraz nowego, mniej popularnego modelu architektury ViT pozwoli na porównanie nie tylko wydajności samych modeli, ale może dostarczyć ciekawych wniosków dotyczących typów architektur oraz ich skuteczności. Każdy z poniższych podrozdziałów przedstawia jedną z rodzin oraz krótko opisuje zasadę ich działania, która odróżnia go od innych.

4.1. MobileNet

MobileNet to rodzina modeli konwolucyjnych sieci neuronowych(CNN). Stworzona przez firmę Google, zaprojektowana została z myślą o wykonywaniu zadań z dziedziny wizji komputerowej - klasyfikacja obrazów, detekcja obiektów itp. Są to małe modele o niskiej liczbie parametrów i małych opóźnieniach, zaprojektowane z myślą o ograniczeniach zasobów obliczeniowych, spotykanych w wielu sprzętach elektronicznych, głównie telefonach mobilnych. Charakterystyczną cechą tego modelu jest zastosowanie splotów separowanych głębinowo. Jest to połączenie splotu głębinowego, który polega na wykonywaniu splotu niezależnie dla każdego kanału mapy cech, co oznacza, że każdy kanał posiada swoje własne jądro, oraz splotu punktowego, który służy do łączenia cech i zmiany wymiaru.[27]

4.2. Transformer wizyjny - ViT

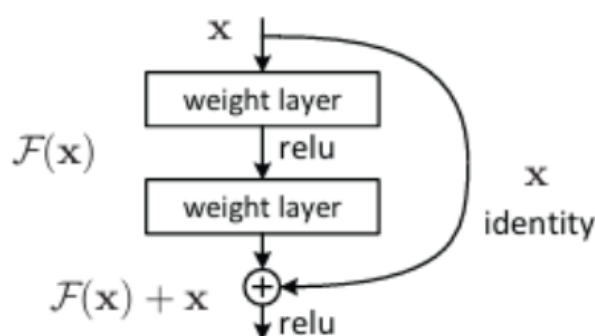
Transformer wizyjny to architektura działająca oddmienne od CNN. Obraz dostarczony do modelu jest dzielony na kawałki, tak zwane patche, o z góry narzuconym rozmiarze. Każdy z patchy jest płaszczony, a następnie mapowany do wektora. Dodawana do wektora jest pozycja patchy, a otrzymany wynik jest przepuszczany przez enkoder transformera. Przegląd modelu znajduje się na . Transformer wizyjny, dzięki swojej unikalnej strukturze, lepiej tworzy powiązania globalne między różnymi elementami na obrazku niż klasyczne podejście CNN.[28]



Rysunek 4.1. Przedstawienie budowy oraz działania ViT.[28]

4.3. ResNet

ResNet to rodzina modeli konwolucyjnych sieci neuronowych (CNN), stworzona przez firmę Microsoft, implementujących koncept nauczania rezydualnego. Rozwiązuje to problem zanikającego/eksplodującego gradientu i umożliwia trenowanie bardzo głębokich sieci neuronowych. W sieci tej użyta jest technika zwana połączeniem skrótowym. Połączenie to łączy aktywacje warstwy do kolejnych warstw, pomijając część warstw. Tworzy to strukturę nazwaną blokiem rezydualnym. Pozwala to sieci na naukę różnicy między wyjściem a wejściem bloku, zamiast dokonywać aproksymacji docelowej funkcji. [22]

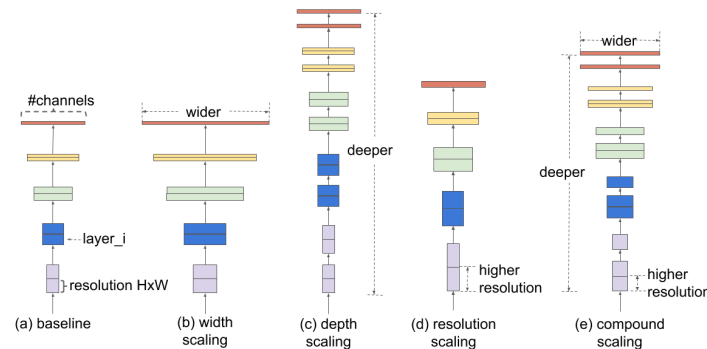


Rysunek 4.2. Przedstawienie budowy oraz działania bloku rezydualnego.[22]

4.4. EfficientNet

EfficientNet to rodzina modeli konwolucyjnych sieci neuronowych (CNN) stworzonych przez Google. Naukowcy stojący za stworzeniem tego modelu, zauważyli, że różne

wymiary skalowania nie są od siebie niezależne. Dla przykładu, dla obrazów o wyższej rozdzielczości, zwiększenie głębokości sieci pomaga polom recepcyjnym w uchwyceniu podobnych cech. Analogicznie, powinno się również zwiększyć szerokość sieci, aby umożliwić znajdowanie bardziej złożonych wzorców w obrazach o większej rozdzielczości. Doprowadziło to autorów do opracowania skalowania złożonego, czyli jednorodnego skalowania szerokości, głębokości oraz rozdzielczości sieci. Pozwoliło to na uzyskanie modelu, który osiąga wyniki lepsze niż inne modele dostępne na rynku w trakcie pierwotnego badania.[29]



Rysunek 4.3. Przykładowa macierz pomyłek dla trzech klas, z wynikami otrzymywanymi dla klasy 'B'.

4.5. DenseNet

DenseNet to rodzina modeli konwolucyjnych sieci neuronowych (CNN), stworzona przez naukowców z Uniwersytetu Kalifornijskiego w Merced. Podobnie jak w modelach ResNet, zaimplementowano tutaj formę połączeń skrótych między warstwami, jednak w różnej formie. Celem twórców było zaprojektowanie architektury, która sprosta występującemu problemowi w czasach tworzenia - zanikającego gradientu. Twórcy tym samym dążyli do redukcji czasu klasyfikacji, aby model mógł być wprowadzony do użytku w ramach analizy wideo w czasie rzeczywistym. W DenseNet, każda warstwa otrzymuje wszystkie wyniki z poprzedzających ją warstw w postaci tensora. Dzięki temu model na każdej warstwie ma dostęp do całości danych, od surowych wejściowych, po te przetworzone przez kolejne warstwy, a nie jak w normalnej sieci konwolucyjnej, czyli tylko wynik z poprzedzającej warstwy. Pozwala to na dodawanie nowych cech, bez tracenia wcześniej wyuczonych informacji.

4.6. ConvNeXt

ConvNeXt to rodzina modeli konwolucyjnych sieci neuronowych (CNN), wykorzystująca rozwiązania zastosowane w ramach transformera wizyjnego. Dane na wejściu przechodzą przez warstwę konwolucyjną, której jądro ma mniejsze wymiary niż w innych CNN, oraz przejścia jądra są równe jego boku. Takie parametry tej warstwy tworzy kawałki

obrazu, które na siebie nienachodzą, wzorując się na patchach w transformerze wizyjnym. Model składa się z cztero etapowej struktury, gdzie każda mapa cech jest przetwarzana w coraz mniejszej rozdzielczości. Modele te stosują także warstwę globalnego uśredniania (z ang. global average pooling - GAP), która zmienia wyjście z warstw konwolucyjnych w jeden wektor. Dodatkowo, zamiast zwykle stosowanej normalizacji po batchach, w tym modelu stosowana jest normalizacja po warstwach, która jest bardziej kompatybilna z architekturą transformera. Całość tworzy model, który jest lżejszy, ma mniej warstw i wymaga mniejszej mocy obliczeniowej.

5. Zbiór danych

5.1. Charakterystyka zbioru danych

Wybrany zbiór danych do wykorzystania w ramach badania to PlantVillage-Dataset. Jest to zbiór danych wykorzystywany w ramach badań przedstawionych w przeglądzie literatury. Zawiera on 54 305 zdjęć liści roślin uprawnych wykonanych w środowisku laboratoryjnym. Dataset ten posiada klasy, które przedstawiają zdrowe okazy różnych gatunków roślin uprawnych, oraz klasy które przedstawiają choroby trawiące te rośliny, razem 38. Zdjęcia te zostały wykonane w warunkach laboratoryjnych, co może nie pełni odzwierciedlać rzeczywiste warunki takie jak światło, czy tło jakie występują w naturze. Niemniej jednak, pozwala on na przeprowadzenie kontrolowanej analizy i zrozumienie badanych zależności oraz na porównanie wyników z badaniami przedstawionymi w ramach przeglądu literatury



Rysunek 5.1. Przykładowe zdjęcia ze zbioru danych.

5.2. Analiza zestawu

Klasa o najmniejszej ilości zdjęć	Potato___healthy
Klasa o największej ilości zdjęć	Orange___Haunglongbing_(Citrus_greening)
Największa ilość zdjęć dla klasy	5507
Najmniejsza ilość zdjęć dla klasy	152
Odchylenie standardowe	1254.8938177261505
Średnia zdjęć na klasę	1429.078947368421

Tabela 5.1. Statystyki liczby zdjęć w zbiorze danych

Analiza statystyczna sugeruje niezerównoważony rozkład zdjęć w zbiorze danych. Jak widać z tabeli 5.1, istnieje duża różnica ilościowa między klasą o największej ilości zdjęć - Orange_Haunglongbing_(Citrus_greening), a klasą o najmniejszej ilości zdjęć - Potato_healthy. Dodatkowo występuje duże odchylenie standardowe, bardzo bliskie średniej ilości zdjęć na klasę. Wszystko to wskazuje na potrzebę zastosowania augmentacji danych, w celu zapobiegnięcia overfittingu.

6. Metodyka badań

Celem poniższego rozdziału jest przedstawienie specyfikacji sprzętowej, metod, narzędzi oraz przeprowadzonych badań porównawczych modeli głębokiego nauczania.

6.1. Specyfikacja sprzętowa

Wszystkie szkolenia oraz testowanie modeli zostały wykonane na sprzęcie o tych samych specyfikacjach, w celu zachowania obiektywności eksperymentu. W badaniach wykorzystano komputer osobisty z konkretnymi specyfikacjami:

- Karta graficzna
 - Model: NVIDIA GeForce RTX 3050 Ti Laptop GPU
 - Pamięć VRAM: 4 GB
- Procesor CPU:
 - Model: AMD Ryzen 5 5600H with Radeon Graphics
 - Liczba rdzeni fizycznych CPU: 6
 - Wątki logiczne: 12
- System operacyjny:
 - Nazwa systemu: Windows 11 Home
 - Wersja systemu: 21H2
 - Typ systemu: 64-bitowy system operacyjny
- Pamięć RAM : 16,0 GB

6.2. Lista modeli wykorzystana w badaniu

Modele zostały wybrane z rodzin opisanych w roz., w celu możliwości analizy szerokiego spektrum rozwiązań dostępnych publicznie, różniących się pod względem architektury, cech oraz celów autorów przyświecających ich zaprojektowaniu. Poniżej przedstawiono listę modeli wybranych do przeprowadzenia badania:

Rodzina modeli	Nazwa modelu	Liczba parametrów	Typ architektury
ResNet	ResNet50	1	CNN
ResNet	ResNet50v2	2	CNN
ViT	ViT-S/16	21,677,214	ViT
ViT	ViT-B/16	4	ViT
MobileNet	MobileNetV2	2,929,246	CNN
MobileNet	MobileNetV3Small	6	CNN
EfficientNet	EfficientNetB0	7	CNN
EfficientNet	EfficientNetB3	8	CNN
DenseNet	DenseNet121	8	CNN
DenseNet	DenseNet169	8	CNN
ConvNeXt	ConvNeXt Small	8	CNN x ViT
ConvNeXt	ConvNeXt Base	8	CNN x ViT

Tabela 6.1. Lista modeli wykorzystanych w badaniu

6.3. Podział zbiorów danych

W ramach badania oryginalny zbiór danych został podzielony na trzy podzbiory: zbiór treningowy, zbiór walidacyjny oraz zbiór testowy. Następnie zbiór treningowy został wykorzystany do stworzenia kolejnych zbiorów. Zostały utworzone zbiory zawierające coraz mniejszą liczbę danych, oraz zbiory zawierające coraz więcej danych z rozmyciem ruchu (z ang. motion blur) oraz szumem gaussowskim (z ang. Gaussian noise).

Zbiór	Liczba zdjęć
Treningowy - 100 % danych	42 429
Treningowy - 50 % danych	21 214
Treningowy - 25 % danych	10 849
Treningowy - 25 % danych zanieczyszczonych	42 429
Treningowy - 50 % danych zanieczyszczonych	42 429
Treningowy - 75 % danych zanieczyszczonych	42 429
Testowy	5 459
Walidacyjny	5 417

Tabela 6.2. Lista zbiorów oraz ilość zdjęć

6.3.1. Dane z różnym procentem uszkodzonych danych

W ramach jednego z eksperymentów, zbiór treningowy został wykorzystany do przygotowania trzech nowych zbiorów treningowych zawierających coraz większy procent danych, które zostały poddane przekształceniom mającym na celu symulację słabszej jakości zdjęć. Zdjęcia zostały poddane poniższym przekształceniom:

- Rozmycie ruchu (z ang. motion blur) - efekt wizualny, który pojawia się kiedy robiąc zdjęcie obiektowi, ten wykonuje ruch. Objawia się to smugami i lekko rozmytym wyglądem,
- Szum gaussowski (z ang. gaussian noise) - to losowy szum podlegający rozkładowi normalnemu, czyli jego wartości skupiają się wokół średniej.

Poniżej przedstawiono przykładowe zdjęcia ze zbioru, na których dokonano przekształceń.



Rysunek 6.1. Przykładowe zdjęcia ze zbioru danych na których dokonano przekształceń.

6.4. Strategia treningu

W ramach treningu dokonano fine-tunningu osiemu wcześniej przedstawionych modeli. Aby otrzymać obiektywne między modelami wyniki testów, modele zostały wyszkolone w ten sam sposób. Każdy z modeli działał na tym samym zbiorze treningowym, testowym oraz walidacyjnym, w ramach jednego eksperymentu. Dla każdego szkolenia zastosowano tę samą konfigurację, a długość szkolenia, choć z góry ustalona, była też kontrolowana techniką EarlyStopping. Podejście to pozwoliło na obiektywną ocenę skuteczności poszczególnych modeli, oraz wyciągnięcie wniosków z poszczególnych eksperymentów.

6.5. Konfiguracja modeli

Niniejszy podrozdział przedstawia ogólną konfigurację modeli wykorzystaną w przeprowadzonym eksperymencie. Opisano w nim przeprowadzoną augmentację danych, wybrane parametry oraz zastosowane techniki w ramach przeprowadzonych eksperymentów. Zastosowanie jednolitej konfiguracji pozwala na bardziej rzetelne i obiektywniejsze porównanie badanych modeli.

6.5.1. Augmentacja danych

Z powodu niezbalansowanego zbioru danych zastosowano augmentację danych, której celem jest zwiększenie różnorodności danych treningowych, co prowadzi do zapobiegania przeuczeniu. Zdjęcia dostarczane do modeli w ramach zbioru treningowego są poddawane przekształceniom, takim jak:

- Losowe odwracanie - zdjęcia zostały lekko obrócone
- Odwracanie poziome - zdjęcia zostały obrócone poziomo tzw. odbicie lustrzane
- Losowy kontrast - zdjęciom zostało losowo podbity kontrast

- Losowe przybliżenie - zdjęcia zostały losowo przybliżone, co prowadzi do przycięcia niektórych elementów obiektu

Warstwa ta działa tylko podczas szkolenia i nie wpływa na obrazy przekazane w celu sprawdzenia skuteczności modelu, czy w celu klasyfikacji. Poniżej przedstawiono przykładowy wygląd losowego zdjęcia, po dokonaniu losowych przekształceń przez warstwę przedstawioną w kodzie ??.



Rysunek 6.2. Przykładowe zdjęcie ze zbioru danych po poddaniu go augmentacji danych.

6.5.2. Parametry

Parametry w procesie treningowym zostały ujednolicone dla każdego modelu, aby ograniczyć wpływ zmiennych zakłócających i zapewnić porównywalność i wiarygodność uzyskanych wyników. Poniższe parametry zostały wybrane również ze względu na ograniczenia sprzętowe:

- Optimizer: Adam
- Epochs: 50
- Batch_size: 128
- Rozmiar zdjęć: 224x224
- Learning rate: 0.001
- Seed: 30

6.5.3. Wczesne zatrzymywanie

Wczesne zatrzymywanie (z ang. Early Stopping) to technika zapobiegająca przeuczeniu modeli w trakcie uczenia. Polega ona na monitorowaniu wybranej metryki, w tej pracy jest to "val_loss" - błąd modelu na danych walidacyjnych (z ang. validation loss). W momencie zaobserwowania braku poprawy w wybranym okresie szkolenie jest zatrzymywane, a wagi modelu zapisywane.

6.5.4. Precyzja mieszana

Precyzja mieszana (z ang. mixed precision) to użycie 16 bitowych oraz 32-bitowych typów zmiennoprzecinkowych w modelu podczas uczenia, w celu przyspieszenia jego działania oraz ograniczenia zużycia pamięci. Pozwala to na skrócenie czasu kroku, przy zachowaniu jakości szkolenia. Standardowo, większość modeli szkolonych jest przy użyciu 32-bitowych liczb zmiennoprzecinkowych, co pozwala na osiągnięcie wysokiej skuteczności, co idzie jednak z kosztem dużego użycia pamięci oraz długiego czasu szkolenia. Precyzja mieszana natomiast używa jednocześnie 16 bitowych oraz 32 bitowych liczb zmiennie przecinkowych. 32 bitowe liczby są używane do krytycznych operacji, które wymagają dużej precyzji, a 16-bitowe do operacji, gdzie mniejsza precyzja jest akceptowalna. Zastosowanie tej techniki pozwala na skrócenie czasu szkolenia oraz zmniejszenie zużycia pamięci, bez wpływu na jakość przeprowadzonego szkolenia.

6.6. Wykonane eksperymenty

6.6.1. Szkolenie na całym zbiorze

Każdy z 12 modeli został wyszkolony w ten sam, wyżej opisany sposób, na pełnym zbiorze danych. Eksperyment ten ma na celu pokazanie skuteczności modeli przy jednakowym zbiorze danych oraz pokazać ich szybkość obliczeniową oraz wielkość.

6.6.2. Szkolenie na zbiorach z różnymi ilościami danych

Każdy z 12 modeli został wyszkolony 3 razy. Za pierwszy raz użyto pełnego zbioru danych, za drugim razem użyto tylko 50% zdjęć ze zbioru, a za trzecim wykorzystano zaledwie 25%. Eksperyment ten ma celu pokazanie skuteczności modeli przy różnych ilościach danych.

6.7. Szkolenie na zbiorach z różnym procentem uszkodzonych danych

Każdy z 12 modeli został wyszkolony 4 razy. Za pierwszym razem użyto czystego zbioru danych, za drugim razem użyto danych szkoleniowych, których 25% zostało poddanych rozmyciu gaussowskiemu, za trzecim 50%, a za czwartym 75%. Celem eksperymentu, było sprawdzenie skuteczności modeli, kiedy do zestawu szkoleniowego dostały się zdjęcia, których jakość może zaszkodzić procesowi szkolenia.

6.8. Metryki ewaluacji wyników

Ocena skuteczności modeli klasyfikacji obrazów wymaga zastosowania wielorakich metryk ewaluacyjnych w celu uzyskania obiektywnego wyniku skuteczności i jakości modelu przez nas badanego. Odpowiedni dobór metryk pozwala na szeroką interpretację otrzymanych wyników, także przy obecności niezbalansowanych danych. Pozwalają na analizę ogólną modelu, a także na identyfikację błędów, takich jak nieodpowiedni dobór parametrów, czy zbioru szkoleniowego. Poniżej przedstawiono szereg metryk ewaluacyjnych wybranych z zamiarem użycia ich w badaniu, wraz ze wzorami oraz krótkim opisem użycia. Są to metryki szeroko stosowane w innych pracach z badanej tematyki, co pozwoli dodatkowo porównać otrzymane wyniki do innych prac w dziedzinie.

6.8.1. Macierz pomyłek

Macierz pomyłek (ang. confusion matrix) stanowi podstawową metrykę ewaluacyjną modeli klasyfikacyjnych, której wyniki są używane następnie do wyliczenia bardziej złożonych metryk pozwalających na obiektywniejszą ocenę modelu. Jest to tabela krzyżowa pozwalająca na porównanie wyników predykcji do oryginalnego przypisania klas do danych. Dzieli ona wynik klasyfikacji na cztery kategorie: TP(true positive), TN(true negative), FP(false positive) i FN(false negative) [30]. Dla klasyfikacji wieloklasowej, macierz pomyłek, jest macierzą o wymiarach $n \times n$, gdzie n to liczba klas, co przedstawiono w przykładzie dla klasy 'B' poniżej.

		Klasy przewidziane przez model			
		Klasy	A	B	C
Prawdziwa klasyfikacja	A	TN	FP	TN	
	B	FN	TP	FN	
	C	TN	FP	TN	

Rysunek 6.3. Przykładowa macierz pomyłek dla trzech klas, z wynikami otrzymywanymi dla klasy 'B'.

6.8.2. Dokładność

Wzór na Dokładność (ang. Accuracy) dzieli sumę poprawnie przypisanych przez model klas (TP i TN) przez sumę wszystkich dokonanych klasyfikacji (TP + TN + FN + FP). [30]

$$Accuracy = \frac{TP + TN}{TP + TN + FN + FP}$$

Jest więc to prawdopodobieństwo dokonania poprawnej predykcji przez model. Jest to podstawowa metryka, pozwalająca na szybką ocenę skuteczności modeli, zwracająca skuteczność predykcji modelu na całym zbiorze danych, nie bierze jednak pod uwagę nie-zbalansowania zbioru danych. Jest to jednak prosta w zrozumieniu metryka pozwalająca na szybka i wstępną ocenę modelu.

6.8.3. Precyzja

Precision to miara informująca nas, jaki procent przypisań danej klasy dokonanych przez model, zgadza się z oryginalnymi przypisaniami. Innymi słowy, z jaką precyzją model dokonuje predykcji danej klasy. [30]

$$Precision = \frac{TP}{TP + FN}$$

Miara ta jest używana dla konkretnych klas, jednak jej średnia, może zostać użyta do oceny całego modelu. Plusem jednak wyliczenia dla każdej klasy tej miary jest szersza ocena skuteczności modelu, co skutkować może wykryciem np. niezbalansowania klasy

6.8.4. Czułość

Czułość (ang. recall) to miara informująca nas, jaki procent danych, które oryginalnie mają daną klasę, zostało poprawnie sklasyfikowanych przed model. Jest więc to liczba poprawnych przypisań danej klasy (TP) podzielona przez liczbę danych mającej tą klasę w zbiorze (TP + FN). [30]

$$Recall = \frac{TP}{TP + FN}$$

Miara ta pozwala na stwierdzenie, czy model jest w stanie poprawnie przypisać klasę do danych. Możliwe jest też użycie średniej tej metryki, co pozwala na uzyskanie oceny całego modelu, jednak pozbawia nas możliwości zbadania klasyfikacji poszczególnych klas.

6.8.5. Miara F1

Miara F1 (ang. F1-Score) jest próbą znalezienia pojedynczej metryki, pozwalającej na jak najbardziej obiektywną ocenę modelu. Metryka agreguje metryki Precyzji i Czułości z pomocą miary harmonicznej. [30]

$$F1 - Score = 2 * (\frac{precision * recall}{precision + recall})$$

Przedstawiony wyżej wzór jest używany tylko w ramach klasyfikacji binarnej. Aby użyć tej miary w ramach klasyfikacji wieloklasowej należy, obliczyć odpowiednie średnie wcześniej wykorzystanych miar. W ramach tego otrzymujemy dwie nowe, ogólne miary: Macro F1-Score, które wykorzystuje arytmetyczną średnią Precyzji i Czułości, oraz Micro F1-Score – wykorzystująca średnie harmoniczną.[30]

$$MacroF1 - Score = 2 * (\frac{ArithmeticPrecision * ArithmeticRecall}{ArithmeticPrecision + ArithmeticRecall})$$

$$MicroF1 - Score = 2 * (\frac{HarmonicPrecision * HarmonicRecall}{HarmonicPrecision + HarmonicRecall})$$

7. Wyniki

7.1. Jakość klasyfikacji modeli

Poniższa tabela prezentuje wyniki uzyskane podczas szkolenia modeli na jednakowym zbiorze zawierającym wszystkie dane oraz jednakowej konfiguracji.

Nazwa modelu	Macro F1-Score	Micro F1-Score	Accuracy	Recall	Precision
ResNet50	0.786	0.840	0.840	0.840	0.840
ResNet50v2	0.961	0.973	0.973	0.973	0.973
ViT-S/16	1	2	3	4	5
ViT-B/16	1	2	3	4	5
MobileNetV2	0.941	0.953	0.953	0.953	0.953
MobileNetV3Small	0.983	0.987	0.987	0.987	0.987
EfficientNetB0	1	2	3	4	5
EfficientNetB3	1	2	3	4	5
DenseNet121	1	2	3	4	5
DenseNet169	1	2	3	4	5
ConvNeXt Small	1	2	3	4	5
ConvNeXt Base	1	2	3	4	5

Tabela 7.1. Wyniki klasyfikacji dla jednakowego zbioru danych

7.2. Efektywność obliczeniowa treningu

Poniższa tabela prezentuje wyniki uzyskane podczas szkolenia modeli na jednakowym zbiorze zawierającym wszystkie dane oraz jednakowej konfiguracji.

Nazwa modelu	Średni czas na przetworzenie jednego batch-a	Params	Waga modelu po szkoleniu	FLOPS (GFLOPS)
ResNet50	0.165 s	2	271 MB	4
ResNet50v2	0.174 s	2	270 MB	4
ViT-S/16	1	2	3	4
ViT-B/16	1	2	3	4
MobileNetV2	0.226 s	2	26.8 MB	4
MobileNetV3Small	0.188 s	2	38.3 MB	4
EfficientNetB0	1	2	3	4
EfficientNetB1	1	2	3	4

Tabela 7.2. Statystyki modeli dla tego samego zbioru danych

7.3. Skalowanie efektywności względem ilości danych treningowych

Poniższa tabela prezentuje wyniki uzyskane podczas szkolenia modeli na zbiorach danych o różnej wielkości oraz jednakowej konfiguracji.

Nazwa modelu	Macro F1-Score		
	100% danych	50 % danych	25 % danych
ResNet50	0.9941	0.9941	0.9751
ResNet101	0.9961	0.9961	0.9860
ViT-B/16	0.9928	0.9940	30.9803
ViT-B/32	0.9846	0.9846	0.9705
MobileNetV2	0.9925	0.9925	0.9866
MobileNetV3Small	0.9921	0.9921	0.9775
EfficientNetB0	0.9937	0.9937	0.9855
EfficientNetB1	0.9928	0.9940	0.9869
DenseNet121	1	2	3
DenseNet169	1	2	3
ConvNeXt Small	1	2	3
ConvNeXt Base	1	2	3

Tabela 7.3. Wyniki klasyfikacji dla zbiorów treningowych o różnej wielkości

7.4. Skalowanie efektywności względem ilości zaszumionych danych

Poniższa tabela prezentuje wyniki uzyskane podczas szkolenia modeli na zbiorach o jednakowej wielkości, z różną ilością danych treningowych poddanych przekształceniu oraz jednakowej konfiguracji.

Nazwa modelu	Macro F1-Score			
	czysty	75%	50%	25%
ResNet50	0.9941	0.9941	0.9943	0.9941
ResNet101	0.9961	0.9960	0.9931	0.9927
ViT-B/16	0.9928	0.9890	0.9961	0.9803
ViT-B/32	0.9846	0.9840	0.9905	0.9837
MobileNetV2	0.9925	0.9950	0.9931	0.9967
MobileNetV3Small	0.9921	0.9987	0.9910	0.9905
EfficientNetB0	0.9937	0.9953	0.9956	0.9962
EfficientNetB1	0.9946	0.9947	0.9945	0.9942
DenseNet121	1	2	3	4
DenseNet169	1	2	3	4
ConvNeXt Small	1	2	3	4
ConvNeXt Base	1	2	3	4

Tabela 7.4. Wyniki klasyfikacji dla różnego procenta zaszumienia zbioru treningowego

7.5. Wnioski

8. Aplikacja

9. Podsumowanie

Celem pracy było zbadanie jak wybrane architektury głębokiego uczenia różnią się pod względem skuteczności oraz odporności na redukcję liczby danych treningowych i degradację ich jakości w zadaniu klasyfikacji chorób roślin. Dodatkowo podjęto próbę praktycznego wykorzystania uzyskanych rezultatów poprzez implementację aplikacji mobilnej wykorzystującej jeden z badanych modeli.

W ramach pracy przeprowadzono szereg eksperymentów pozwalających ocenić wpływ jakości oraz ilości danych na skuteczność badanych modeli. Uzyskane wyniki zostały wykorzystane w ramach analizy porównawczej architektur oraz umożliwiły określenie różnic w ich odporności na ograniczenia związane z ilością i jakością danych treningowych. Dokonano przeglądu różnych technik uczenia maszynowego oraz przedstawiono wybrane architektury do badania - ViT, EfficientNet, MobileNet, ResNet, DenseNet oraz ConvNeXt.

Na podstawie przeprowadzonych badań można stwierdzić, że wszystkie architektury biorące udział w badaniu są skuteczne w ramach zadania klasyfikacji chorób roślin.

Przyszłe badania można rozszerzyć o szkolenie na zbiorze danych składającym się głównie ze zdjęć wykonanych w naturalnym środowisku, gdzie występują zmienne warunki oświetlenia, różne tła oraz kąty wykonania zdjęć. Istotne było by także rozszerzenie badań o inne architektury, które nie zostały wzięte pod uwagę w ramach tego badania.

Bibliografia

- [1] Food and Agriculture Organization of the United Nations, *The Hidden Health Crisis: How Plant Diseases Threaten Global Food Security*, <https://www.fao.org/one-health/highlights/how-plant-diseases-threaten-global-food-security/en>, Dostęp: 16.06.2026, 2025.
- [2] Polski Czerwony Krzyż, *Światowy Dzień Walki z Głodem*, <https://pck.pl/polski-czerwony-krzyz/aktualnosci/2025-10-16-swiatowy-dzien-walki-z-glodem>, Dostęp: 16.06.2026, 2025.
- [3] S. P. Mohanty, D. P. Hughes i M. Salathé, „Using Deep Learning for Image-Based Plant Disease Detection”, *Frontiers in Plant Science*, t. 7, 2016.
- [4] M. E. H. Chowdhury i in., „Automatic and Reliable Leaf Disease Detection Using Deep Learning Techniques”, *AgriEngineering*, t. 3, 2021.
- [5] K. G. Liakos, P. Busato, D. Moshou, S. Pearson i D. Bochtis, „Machine Learning in Agriculture: A Review”, *Sensors (Basel)*, t. 18, nr. 8, s. 2674, sierp. 2018. DOI: 10.3390/s18082674. adr.: <https://doi.org/10.3390/s18082674>.
- [6] D. Singh, N. Jain, P. Jain, P. Kayal, S. Kumawat i N. Batra, *PlantDoc: A Dataset for Visual Plant Disease Detection*, Dostęp zdalny (08.06.2025): <https://doi.org/10.1145/3371158.3371196>, Hyderabad, India, 2020. DOI: 10.1145/3371158.3371196.
- [7] T. O. Emmanuel, *PlantVillage Dataset*, Dostęp zdalny (08.06.2025): <https://www.kaggle.com/datasets/emmarex/plantdisease/data?select=PlantVillage>, 2018.
- [8] S. Bhattarai, *New Plant Diseases Dataset*, Dostęp zdalny (08.06.2025): <https://www.kaggle.com/datasets/vipooooool/new-plant-diseases-dataset>, 2018.
- [9] P. AB, *Planta: Plant & Garden Care*, Dostęp zdalny (08.06.2025): <https://play.google.com/store/apps/details?id=com.stromming.planta>, 2025.
- [10] S. Ltd, *Agrio - Plant diagnosis app*, Dostęp zdalny (08.06.2025): <https://play.google.com/store/apps/details?id=com.agrio>, 2025.
- [11] PlantNet, *PlantNet Plant Identification*, Dostęp zdalny (08.06.2025): <https://play.google.com/store/apps/details?id=org.plantnet>, 2025.
- [12] T. M. Mitchell, *Machine Learning*. McGraw-Hill, 1997.
- [13] Y. LeCun, Y. Bengio i G. Hinton, „Deep learning”, *nature*, t. 521, nr. 7553, s. 436–444, 2015.
- [14] TensorFlow Agents Team. „Introduction to RL and Deep Q Networks”. Accessed: 2026-05-19. adr.: https://www.tensorflow.org/agents/tutorials/0_intro_rl.
- [15] C. Cortes i V. Vapnik, „Support-vector networks”, *Machine Learning*, t. 20, s. 273–297, wrz. 1995. DOI: 10.1007/BF00994018. adr.: <https://doi.org/10.1007/BF00994018>.
- [16] A. Horzyk, *Support Vector Machines (SVM)*, <https://home.agh.edu.pl/~horzyk/lectures/miw/MIW-SVM.pdf>, Lecture notes, accessed: 2026-05-19, 2026.

- [17] I. Mienye i N. Jere, „A Survey of Decision Trees: Concepts, Algorithms, and Applications”, *IEEE Access*, t. PP, s. 1–1, sty. 2024. DOI: 10.1109/ACCESS.2024.3416838.
- [18] M. Suyal i P. Goyal, „A review on analysis of k-nearest neighbor classification machine learning algorithms based on supervised learning”, *International Journal of Engineering Trends and Technology*, t. 70, nr. 7, s. 43–48, 2022.
- [19] A. Horzyk, *Sztuczne Sieci Neuronowe MLP*, <https://home.agh.edu.pl/~horzyk/lectures/miw/MIW-SztuczneSieciNeuronoweMLP.pdf>, Dostęp: 2026-05-19, 2026.
- [20] K. Kaczmariski. „MiNI Wykłady”, term. wiz. 19 maj. 2026. adr.: <https://pages.mini.pw.edu.pl/~kaczmariskik/MiNIwyklady/index.html>.
- [21] GeeksforGeeks. „Multi-Layer Perceptron Learning in TensorFlow”. Accessed: 2026-05-19. adr.: <https://www.geeksforgeeks.org/deep-learning/multi-layer-perceptron-learning-in-tensorflow/>.
- [22] K. He, X. Zhang, S. Ren i J. Sun, „Deep residual learning for image recognition”, w *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, s. 770–778.
- [23] KSolves. „Understanding Convolution Neural Network Architecture”. Accessed: 2026-05-19. adr.: <https://www.ksolves.com/blog/artificial-intelligence/understanding-convolution-neural-network-architecture>.
- [24] K. O’Shea i R. Nash, „An Introduction to Convolutional Neural Networks”, *CoRR*, t. abs/1511.08458, 2015. arXiv: 1511.08458. adr.: <http://arxiv.org/abs/1511.08458>.
- [25] A. Vaswani i in., „Attention Is All You Need”, *CoRR*, t. abs/1706.03762, 2017. arXiv: 1706.03762. adr.: <http://arxiv.org/abs/1706.03762>.
- [26] R. Kumar. „Recurrent Neural Network (RNN)”. Accessed: 2026-05-19. adr.: <https://medium.com/@RobuRishabh/recurrent-neural-network-rnn-8412b9abd755>.
- [27] A. G. Howard, „Mobilenets: Efficient convolutional neural networks for mobile vision applications”, *arXiv preprint arXiv:1704.04861*, 2017.
- [28] A. Dosovitskiy, „An image is worth 16x16 words: Transformers for image recognition at scale”, *arXiv preprint arXiv:2010.11929*, 2020.
- [29] M. Tan i Q. Le, „Efficientnet: Rethinking model scaling for convolutional neural networks”, w *International conference on machine learning*, PMLR, 2019, s. 6105–6114.
- [30] M. Grandini, E. Bagli i G. Visani, „Metrics for multi-class classification: an overview”, *arXiv preprint arXiv:2008.05756*, 2020.

Wykaz symboli i skrótów

EiTI – Wydział Elektroniki i Technik Informatycznych

PW – Politechnika Warszawska

WEIRD – ang. *Western, Educated, Industrialized, Rich and Democratic*

Spis rysunków

2.1	Przykładowe zrzuty ekranu z aplikacji Planta.	15
2.2	Przykładowe zrzuty ekranu z aplikacji Agrio.	16
2.3	Przykładowe zrzuty ekranu z aplikacji.	17
3.1	Schemat ilustrujący działanie algorytmu SVM	20
3.2	Schemat budowy drzewa decyzyjnego [17]	21
3.3	Schemat budowy neuronu	22
3.4	Schemat budowy perceptronu	23
3.5	Schemat ilustrujący budowę MLP	24
3.6	Schemat budowy CNN.[23]	25
3.7	Przedstawienie jednostki rekurencyjnej.[13]	27
3.8	Przedstawienie rozwijania RNN	27
4.1	Przedstawienie budowy oraz działania ViT.[28]	29
4.2	Przedstawienie budowy oraz działania bloku rezyduального.[22]	29
4.3	Przykładowa macierz pomyłek dla trzech klas, z wynikami otrzymywanymi dla klasy 'B'.	30
5.1	Przykładowe zdjęcia ze zbioru danych.	32
6.1	Przykładowe zdjęcia ze zbioru danych na których dokonano przekształceń.	35
6.2	Przykładowe zdjęcie ze zbioru danych po poddaniu go augmentacji danych.	36
6.3	Przykładowa macierz pomyłek dla trzech klas, z wynikami otrzymywanymi dla klasy 'B'.	38

Spis tabel

5.1	Statystyki liczby zdjęć w zbiorze danych	32
6.1	Lista modeli wykorzystanych w badaniu	34
6.2	Lista zbiorów oraz ilość zdjęć	34
7.1	Wyniki klasyfikacji dla jednakowego zbioru danych	41
7.2	Statystyki modeli dla tego samego zbioru danych	41
7.3	Wyniki klasyfikacji dla zbiorów treningowych o różnej wielkości	42
7.4	Wyniki klasyfikacji dla różnego procenta zaszumienia zbioru treningowego	43

Spis załączników